

CONTENTS

Loss functions

Performance metrics

Practical methodology

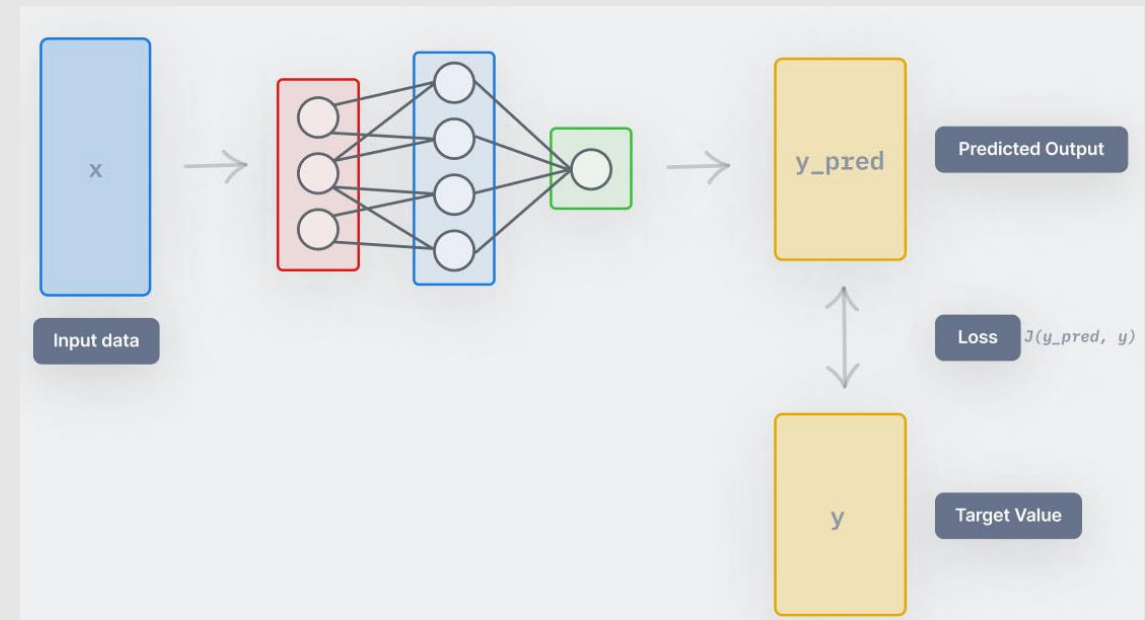
LOSS FUNCTIONS

Loss functions measure **how close** a predicted value is to the actual value.

- Small deviations -> small loss
- Large deviations -> large loss

Mathematical objects that **guide the model training** process towards correct predictions.

- Model parameters that **minimize** the loss of the model on a given training dataset.

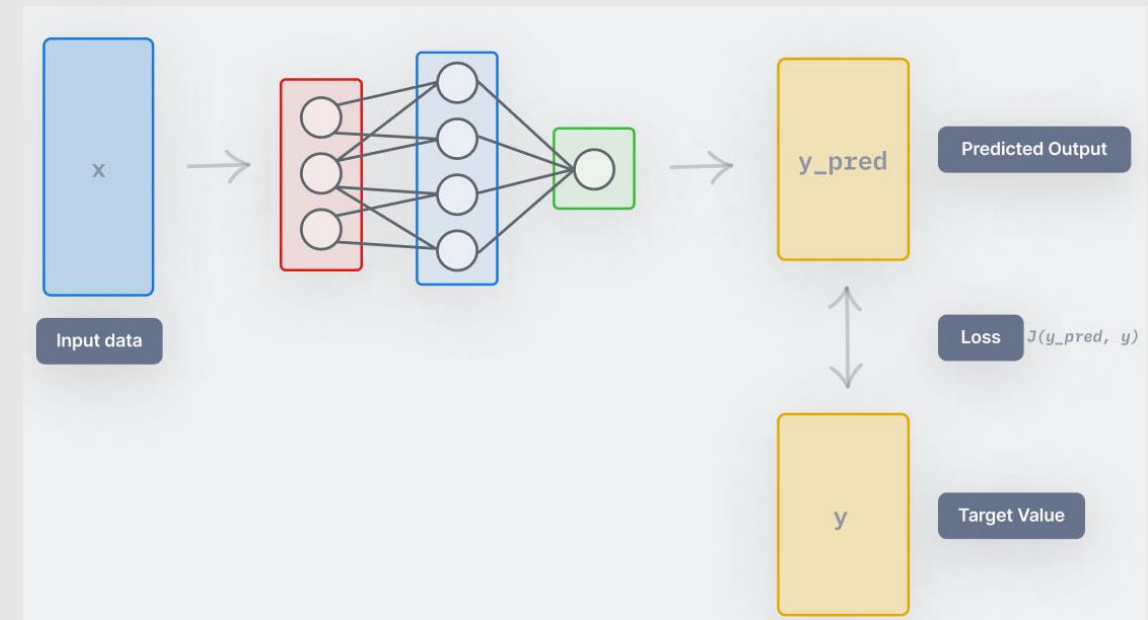


<https://www.v7labs.com/blog/pytorch-loss-functions>

LOSS FUNCTIONS

The form of the loss function depends on the problem at hand (**problem specific**):

- **Regression**
 - L1 Loss (Mean Absolute Error)
 - Smooth L1 Loss
 - Mean Squared Error
- **Classification**
 - Cross Entropy Loss
 - KL Divergence Loss
- **Ranking**
 - Margin Ranking Loss
- **Representation learning**
 - Cosine Embedding Loss
 - Triplet Margin Loss



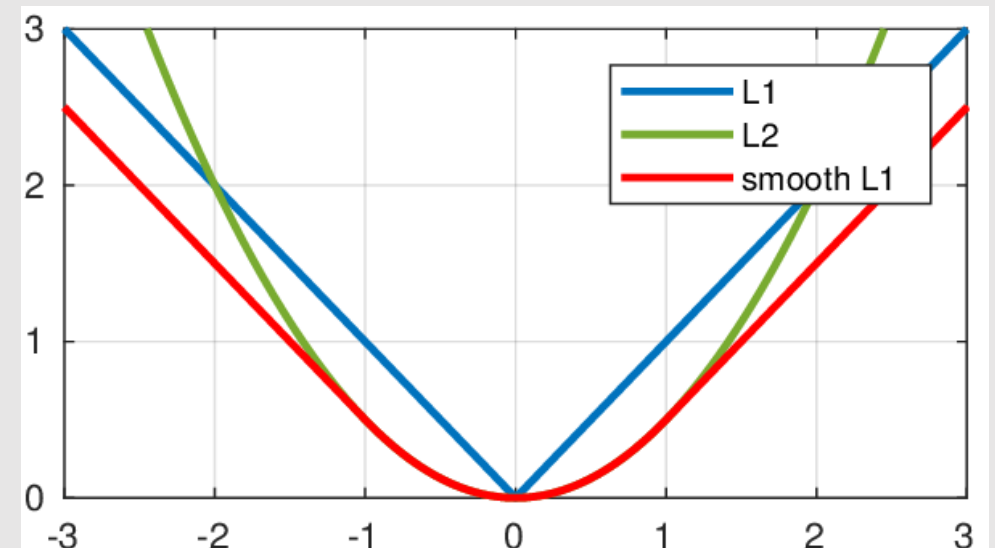
<https://www.v7labs.com/blog/pytorch-loss-functions>

LOSS FUNCTIONS - REGRESSION

L2 Loss (Mean Squared Error) gives the mean squared error between each element of the input x (predictions) and target y (desired output).

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = (x_n - y_n)^2$$

- The most commonly used loss
 - Closed form solutions
- Very sensitive to outliers
 - The error for outlier points is squared
 - Overfit the outliers

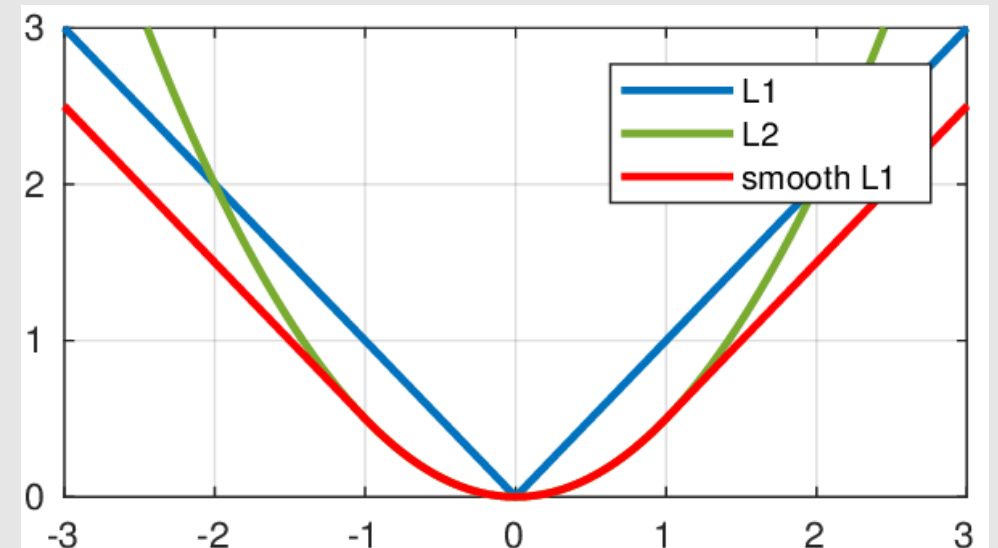


LOSS FUNCTIONS - REGRESSION

L1 Loss (Mean Absolute Error) measures the mean absolute error between each element of the input x (predictions) and target y (desired output).

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = |x_n - y_n|$$

- Robust against outliers and noise
- Not differentiable at 0
 - this complicates the learning process



LOSS FUNCTIONS - REGRESSION

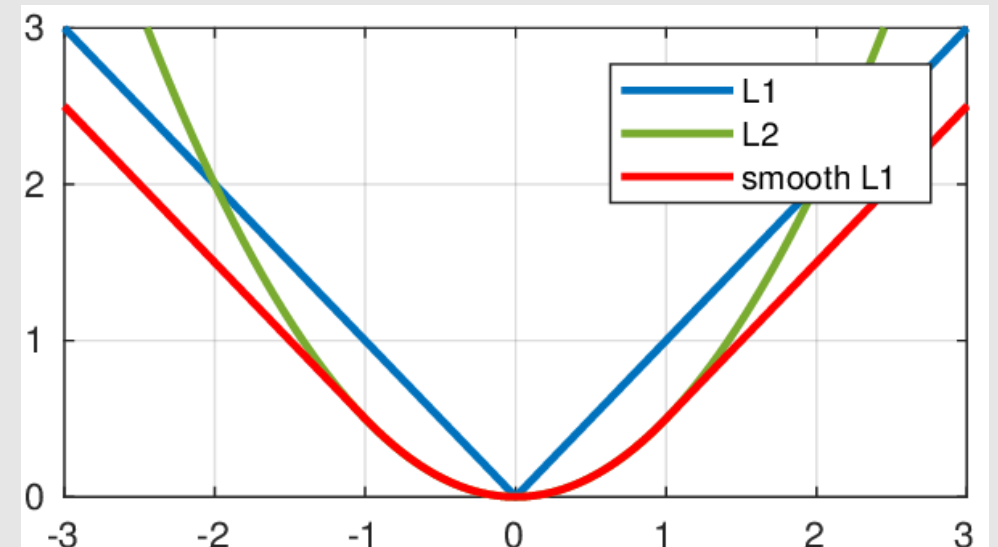
Smooth L1 Loss uses L2 loss when the absolute element wise error falls below 1 and L1 loss otherwise.

$$l(x, y) = \frac{1}{N} \sum_{n=1}^N z_n$$

$$z_n = \begin{cases} 0.5(x_n - y_n)^2, & \text{if } |x_n - y_n| < 1 \\ |x_n - y_n| - 0.5, & \text{otherwise} \end{cases}$$

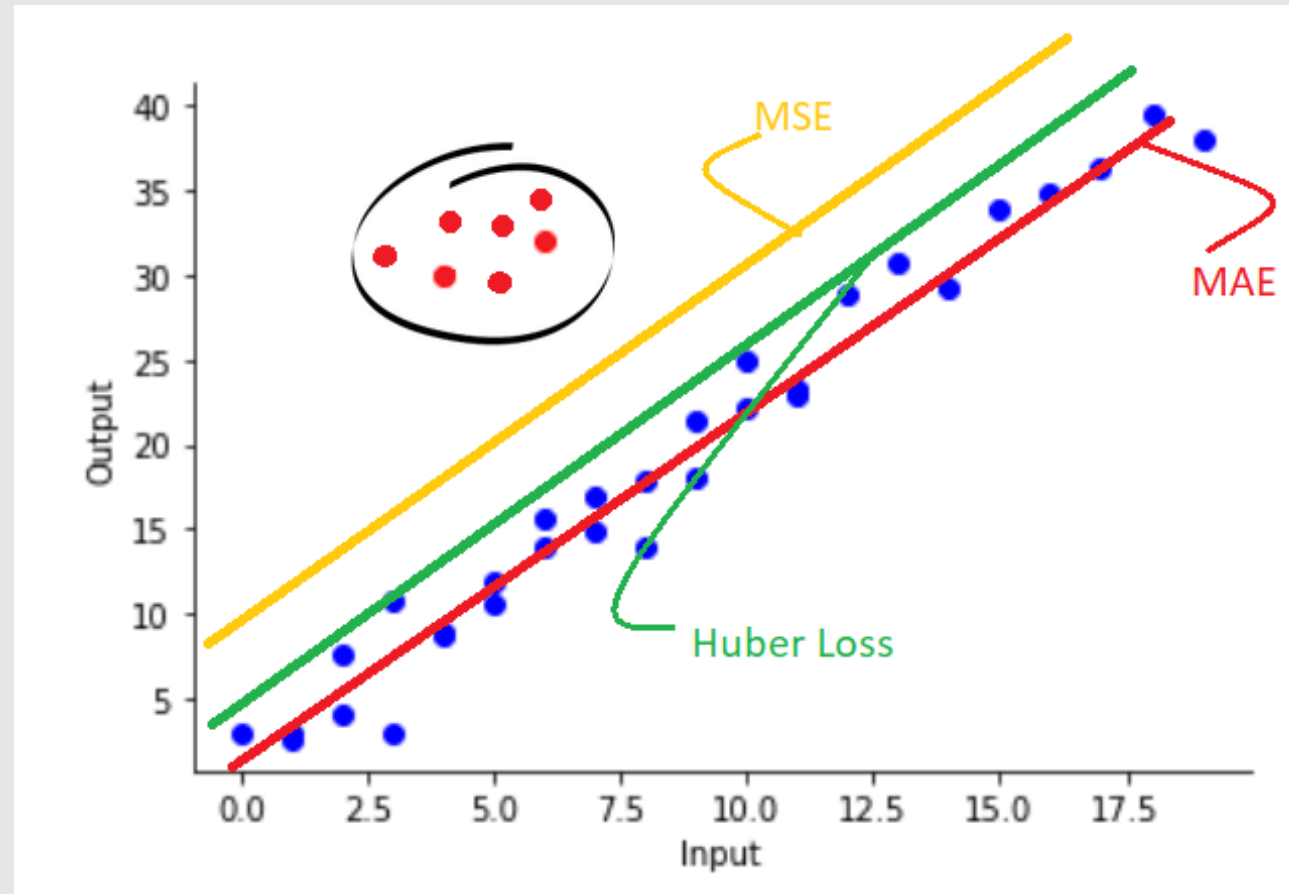
- Combines the advantages of both L1 and L2.

Also known as
Huber Loss



LOSS FUNCTIONS - REGRESSION

Smooth L1 Loss versus L1 Loss versus L2 Loss in the presence of outliers.



LOSS FUNCTIONS - CLASSIFICATION

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

- Penalizes greatly for being confident and wrong

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = - \sum_c w_c \cdot \log \left(\frac{\exp(x_n^{(c)})}{\sum_j \exp(x_n^{(j)})} \right) \cdot y_n^{(c)}$$

LOSS FUNCTIONS - CLASSIFICATION

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

- Penalizes greatly for being confident and wrong

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = - \sum_c \underbrace{w_c}_{\downarrow} \log \left(\frac{\exp(x_n^{(c)})}{\sum_j \exp(x_n^{(j)})} \right) \cdot y_n^{(c)}$$

The weight vector is useful if the frequency is different for each category/class.

LOSS FUNCTIONS - CLASSIFICATION

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

- Penalizes greatly for being confident and wrong

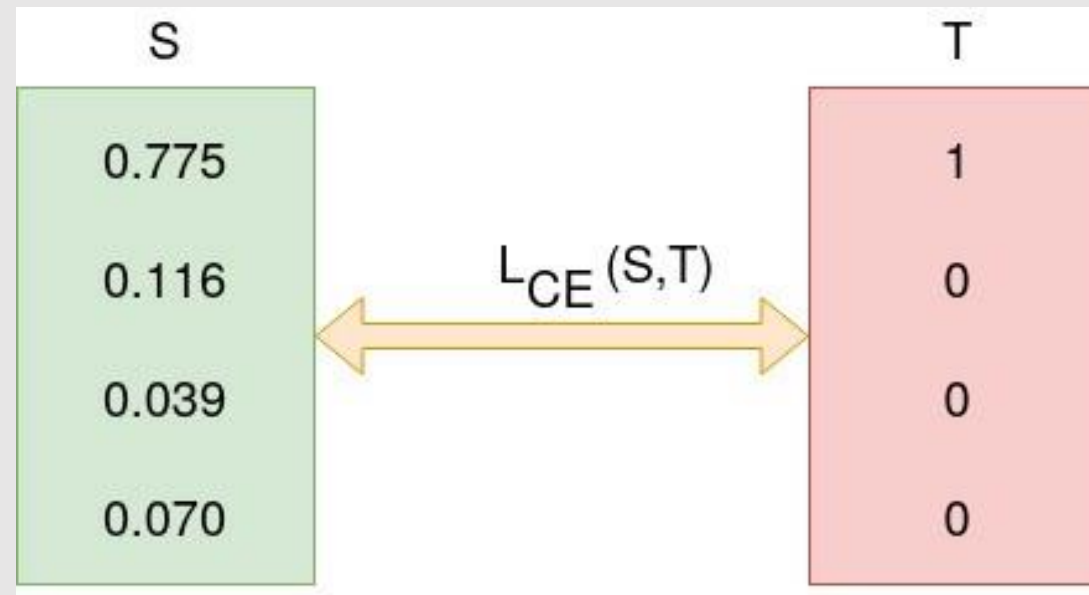
$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = - \sum_c w_c \cdot \log \left(\frac{\exp(x_n^{(c)})}{\sum_j \exp(x_n^{(j)})} \right) \cdot y_n^{(c)}$$

- The predicted distribution is represented by x_n and the actual distribution by y_n .

LOSS FUNCTIONS - CLASSIFICATION

Cross-Entropy Loss is used to learn the probability distribution of the data with respect to the classes they belong.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = - \sum_c w_c \cdot \log \left(\frac{\exp(x_n^{(c)})}{\sum_j \exp(x_n^{(j)})} \right) \cdot y_n^{(c)}$$



LOSS FUNCTIONS - CLASSIFICATION

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = -(y_n \log x_n + (1 - y_n) \log(1 - x_n))$$

LOSS FUNCTIONS - CLASSIFICATION

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = -(\underbrace{y_n \log x_n}_{\substack{\text{Cancels out when} \\ \text{the desired output} \\ \text{is 0}}} + \underbrace{(1 - y_n) \log(1 - x_n)}_{\substack{\text{Cancels out when} \\ \text{the desired output} \\ \text{is 1}}})$$

LOSS FUNCTIONS - CLASSIFICATION

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

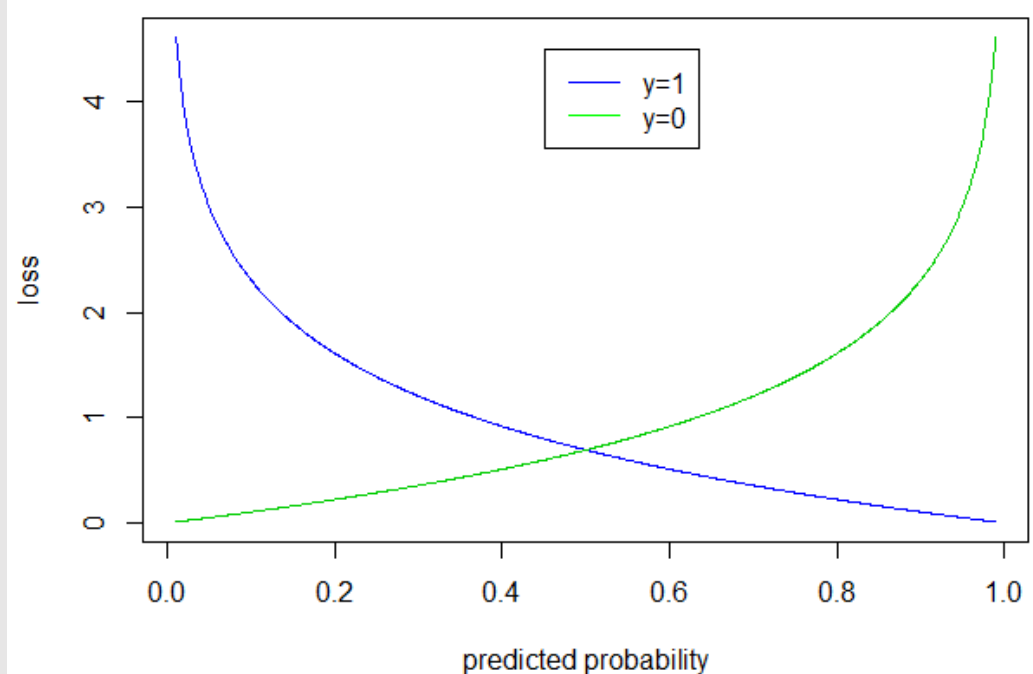
$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = -(\underbrace{y_n \log x_n}_{\substack{\text{Cancels out when} \\ \text{the desired output} \\ \text{is 0}}} + \underbrace{(1 - y_n) \log(1 - x_n)}_{\substack{\text{Cancels out when} \\ \text{the desired output} \\ \text{is 1}}})$$

This formula assumes that x_n and y_n are probabilities, so they are strictly between 0 and 1.

LOSS FUNCTIONS - CLASSIFICATION

Binary Cross-Entropy Loss is similar to cross-entropy loss when the data belong to **two** different classes.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = -(\underbrace{y_n \log x_n}_{\text{Cancels out when the desired output is 0}} + \underbrace{(1 - y_n) \log(1 - x_n)}_{\text{Cancels out when the desired output is 1}})$$



Cancels out when
the desired output
is 0

Cancels out when
the desired output
is 1

This formula assumes that x_n and y_n are probabilities, so they are strictly between 0 and 1.

LOSS FUNCTIONS - CLASSIFICATION

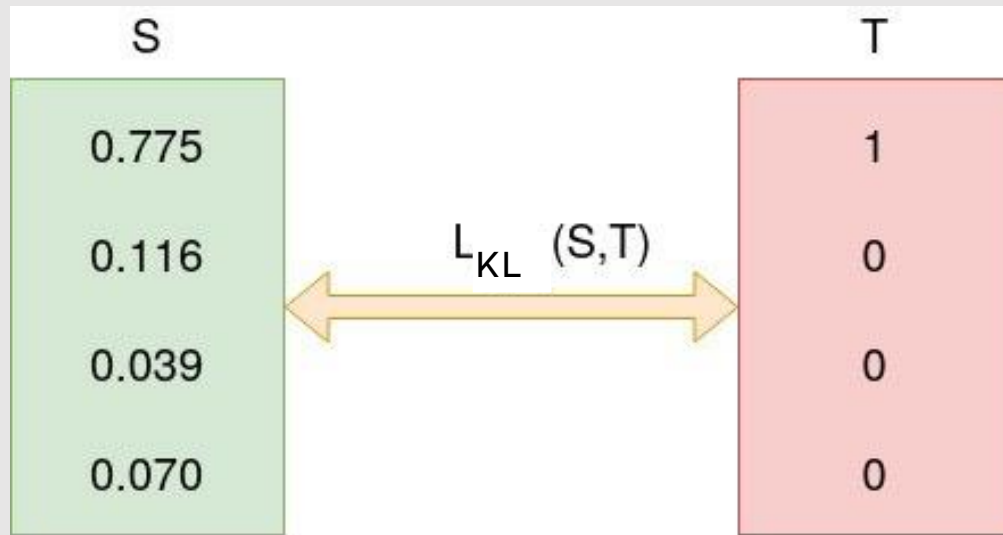
Kullback-Leibler (KL) Divergence measures how two probability distributions are different from each other.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = y_n \log \frac{y_n}{x_n} = y_n (\log y_n - \log x_n)$$

LOSS FUNCTIONS - CLASSIFICATION

Kullback-Leibler (KL) Divergence measures how two probability distributions are different from each other.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = y_n \log \frac{y_n}{x_n} = y_n (\log y_n - \log x_n)$$



Expectation of the
logarithmic difference
between the two
distributions

LOSS FUNCTIONS - CLASSIFICATION

Kullback-Leibler (KL) Divergence measures how two probability distributions are different from each other.

$$l(x, y) = \{l_1, l_2, \dots, l_N\} \text{ where } l_n = y_n \log \frac{y_n}{x_n} = y_n (\log y_n - \log x_n)$$

- Non-negative.
- The same can be achieved with cross entropy with lesser computation.

LOSS FUNCTIONS - RANKING

Margin Ranking Loss measures the relative distance between pairs of inputs belonging to a dataset.

$l(x, y) = \{l_1, l_2, \dots, l_N\}$ where $l_n = \max(0, \Delta - y_n \cdot (x_{n,1} - x_{n,2}))$

- $y_n \in \{-1, 1\}$: -1 if $x_{n,1} \succ x_{n,2}$ and 1 otherwise
- If $y_n \cdot (x_{n,1} - x_{n,2})$ is larger than a margin Δ the loss is zero.
- If $y_n \cdot (x_{n,1} - x_{n,2})$ is smaller than a margin Δ the loss increases linearly.

It is widely used for **preference learning** and **ranking** tasks and tasks that include **subjectively defined** variables.

LOSS FUNCTIONS – REPRESENTATION LEARNING

Triplet Margin Loss measures the relative similarity between samples.

$$l_n(a_n, p_n, n_n) = \max(0, \Delta + d(a_n, p_n) - d(a_n, n_n))$$

- a_n is anchor sample, p_n is a sample that is similar to the anchor and n_n is a sample that is dissimilar to the anchor.
- Sends the first distance toward 0 and the second distance larger than some margin.
- The only thing that matter is that the distance between the good pair is smaller than the distance between the bad pair.

LOSS FUNCTIONS – REPRESENTATION LEARNING

Cosine Embedding Loss measures whether two inputs are similar or dissimilar, using the cosine distance.

$$l_n(x_{n,1}, x_{n,2}) = \begin{cases} 1 - \cos(x_{n,1}, x_{n,2}) & \text{if } x_{n,1} \text{ and } x_{n,2} \text{ are similar} \\ \max(0, \cos(x_{n,1}, x_{n,2}) - \Delta) & \text{otherwise} \end{cases}$$

LOSS FUNCTIONS – REPRESENTATION LEARNING

Cosine Embedding Loss measures whether two inputs are similar or dissimilar, using the cosine distance.

$$l_n(x_{n,1}, x_{n,2}) = \begin{cases} 1 - \cos(x_{n,1}, x_{n,2}) & \text{if } x_{n,1} \text{ and } x_{n,2} \text{ are similar} \\ \max(0, \cos(x_{n,1}, x_{n,2}) - \Delta) & \text{otherwise} \end{cases}$$

- Normalised Euclidean distance.
- Positive cases result to aligned vector representations.
- Negative cases result to orthogonal vector representations.
- Building block for contrastive learning.

LOSS FUNCTIONS – REPRESENTATION LEARNING

Contrastive Loss is used for (un)supervised representation learning.

$$l_n = -\log \frac{\exp(\text{sim}(x_n, p)/\tau)}{\sum_{k=1}^K \exp(\text{sim}(x_n, n_k)/\tau)}$$

- Contrasts an anchor point against a positive (similar) example and again K negative examples.

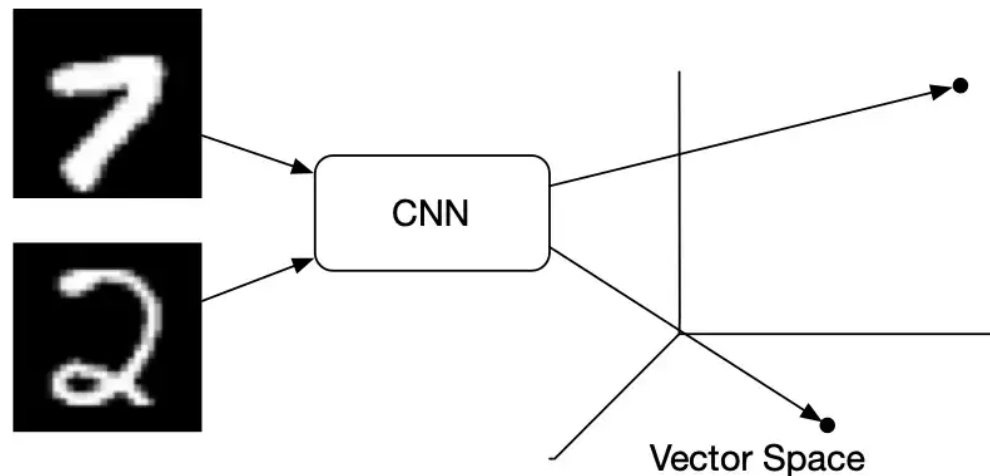
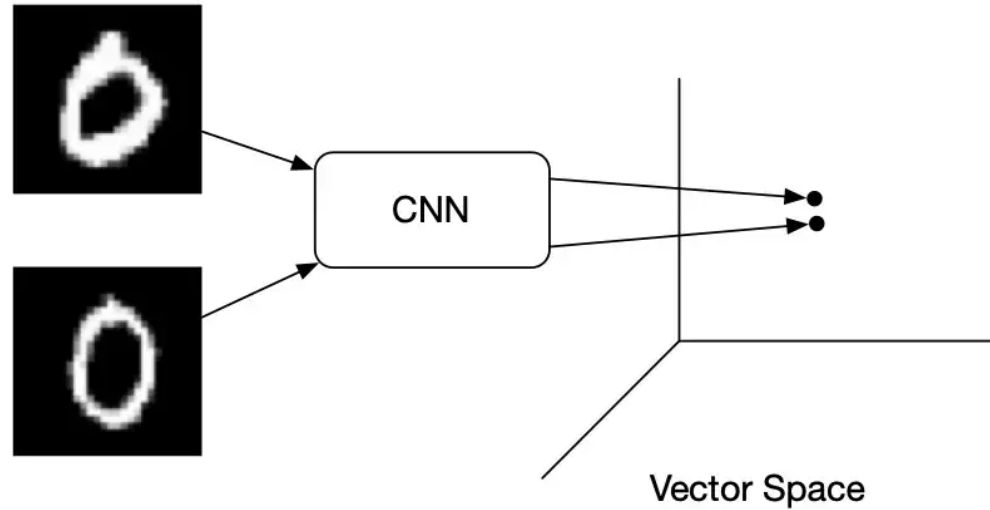
LOSS FUNCTIONS – REPRESENTATION LEARNING

Contrastive Loss is used for (un)supervised representation learning.

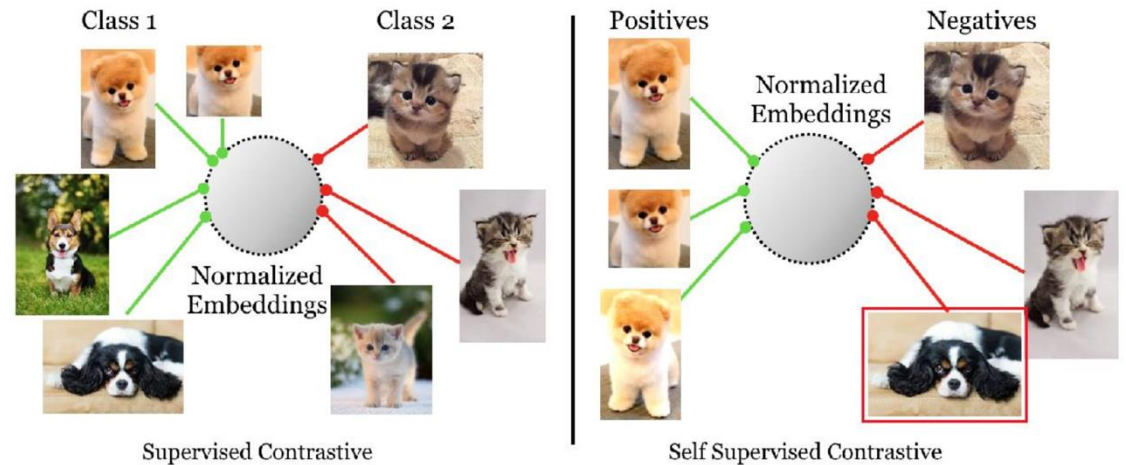
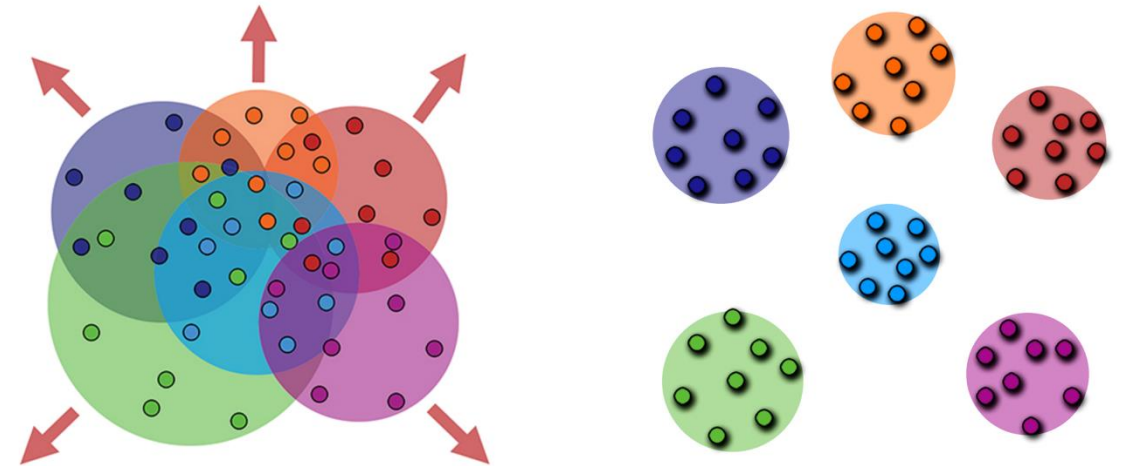
$$l_n = -\log \frac{\exp(\text{sim}(x_n, p)/\tau)}{\sum_{k=1}^K \exp(\text{sim}(x_n, n_k)/\tau)}$$

- Contrasts an anchor point against a positive (similar) example and again K negative examples
- **Unsupervised:** the positive example corresponds to a transformation of x_n and negative examples are all the examples in the dataset except x_n .
- **Supervised:** positive are examples that belong to the same class as x_n and negative are those that belong to different classes.

LOSS FUNCTIONS – REPRESENTATION LEARNING



<https://towardsdatascience.com/contrastive-loss-explained-159f2d4a87ec>

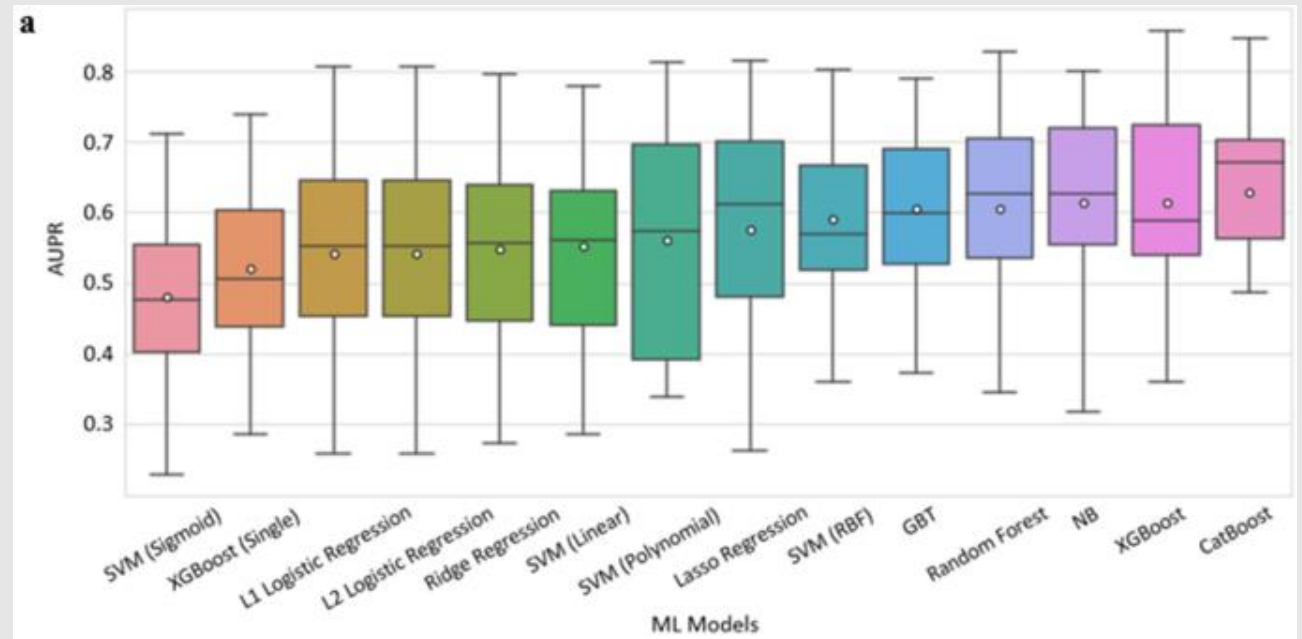


<https://gowrishankar.info/blog/introduction-to-contrastive-loss-similarity-metric-as-an-objective-function/>

PERFORMANCE METRICS

Performance metrics tell you if your models are making progress and put a number on it.

- Are usually different from loss functions
 - Loss functions are used for **training**, while performance metrics for **testing**
- Define our **goals**
- **Problem depended**
 - Regression metrics
 - Classification metrics
 - Ranking metrics
 - Representation learning metrics



PERFORMANCE METRICS - REGRESSION

Mean Squared Error (MSE) and **Mean Absolute Error (MAE)** are perhaps the most popular metric used for regression problems.

- **MSE** finds the **average of the squared difference** between the target value and the value predicted by the regression model.

$$MSE = \frac{1}{N} \sum_{n=1}^N (x_n - y_n)^2$$

- **MAE** finds the **average of the absolute difference** between the target value and the value predicted by the regression model.

$$MAE = \frac{1}{N} \sum_{n=1}^N |x_n - y_n|$$

PERFORMANCE METRICS - REGRESSION

Mean Squared Error (MSE) and **Mean Absolute Error (MAE)** are perhaps the most popular metric used for regression problems.

- **MSE** finds the **average of the squared difference** between the target value and the value predicted by the regression model.

$$MSE = \frac{1}{N} \sum_{n=1}^N (x_n - y_n)^2 \rightarrow \text{MSE units correspond to the squared units of } x_n$$

- **MAE** finds the **average of the absolute difference** between the target value and the value predicted by the regression model.

$$MAE = \frac{1}{N} \sum_{n=1}^N |x_n - y_n| \rightarrow \text{MAE units are the same as the units of } x_n$$

PERFORMANCE METRICS - REGRESSION

R-squared (R^2) provides an indication of the **goodness of fit** of a set of predicted values and actual output values.

$$R^2 = 1 - \frac{\sum_n (y_n - x_n)^2}{\sum_n (y_n - \bar{y})^2}$$

- Compares the fit of the trained model with that of a **horizontal line**, which serves as **baseline**.
- If the model fit is **worse (better)** than that of the horizontal line, then R^2 is **negative (positive)**.
- R^2 is bounded between -1 and 1.

PERFORMANCE METRICS - CLASSIFICATION

Confusion matrix presents a summary of the classification performance in terms of true/false positives and true/false negatives.

		Actual Values	
		Positive (1)	Negative (0)
Predicted Values	Positive (1)	TP	FP
	Negative (0)	FN	TN

PERFORMANCE METRICS - CLASSIFICATION

Accuracy is the fraction of predictions our model got right out of all the predictions.

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- It ranges between zero and one.
- Great for balanced datasets.
- Problematic for imbalanced datasets.
 - Extreme case -> models that are not predictive

PERFORMANCE METRICS - CLASSIFICATION

Precision indicates the proportion of positive predictions that are actually correct.

$$\text{precision} = \frac{TP}{TP + FP}$$

- To have the whole picture, it is used along with recall.
- It overcomes the problem of accuracy when the data are imbalanced.

PERFORMANCE METRICS - CLASSIFICATION

Recall indicates the ratio of actual positives predicted as such against the total number of positive examples.

$$\text{recall} = \frac{TP}{TP + FN}$$

- To have the whole picture, it is used along with precision.
- It overcomes the problem of accuracy when the data are imbalanced.

PERFORMANCE METRICS - CLASSIFICATION

F1-score balances precision and recall. It corresponds to the harmonic mean of both precision and recall.

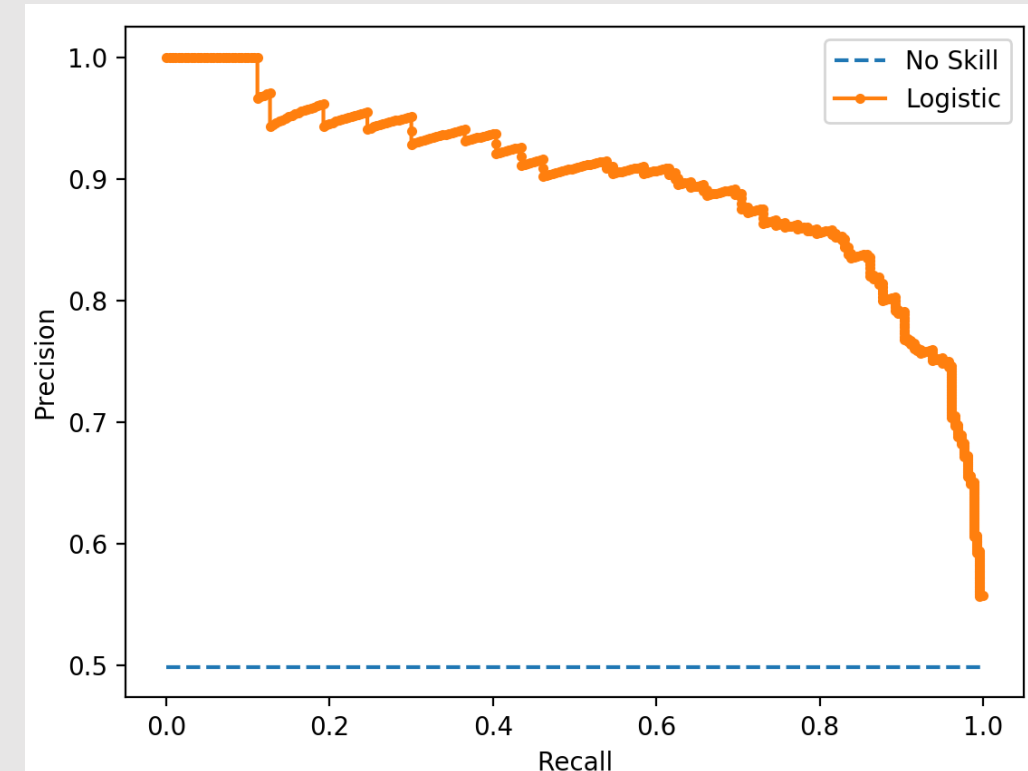
$$\text{F1 score} = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

- It is used when both precision and recall are equally important.
- Overcomes the limitations of accuracy when imbalanced datasets are used.

PERFORMANCE METRICS - CLASSIFICATION

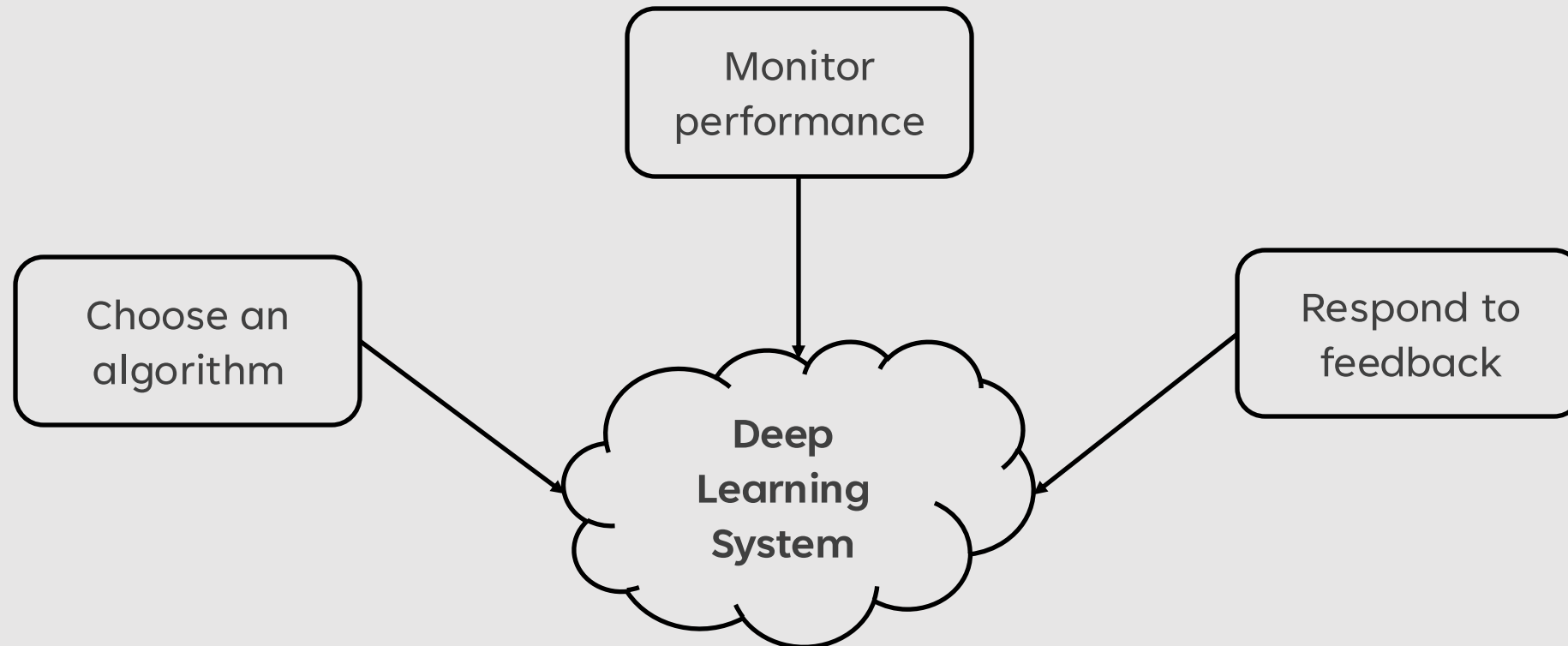
Precision/recall curve plots performance over a y-axis showing precision and an x-axis which is recall.

- Each point is evaluated at different threshold values for $P(\text{positive} \mid x)$.
- Great when the different classes are not equally important.



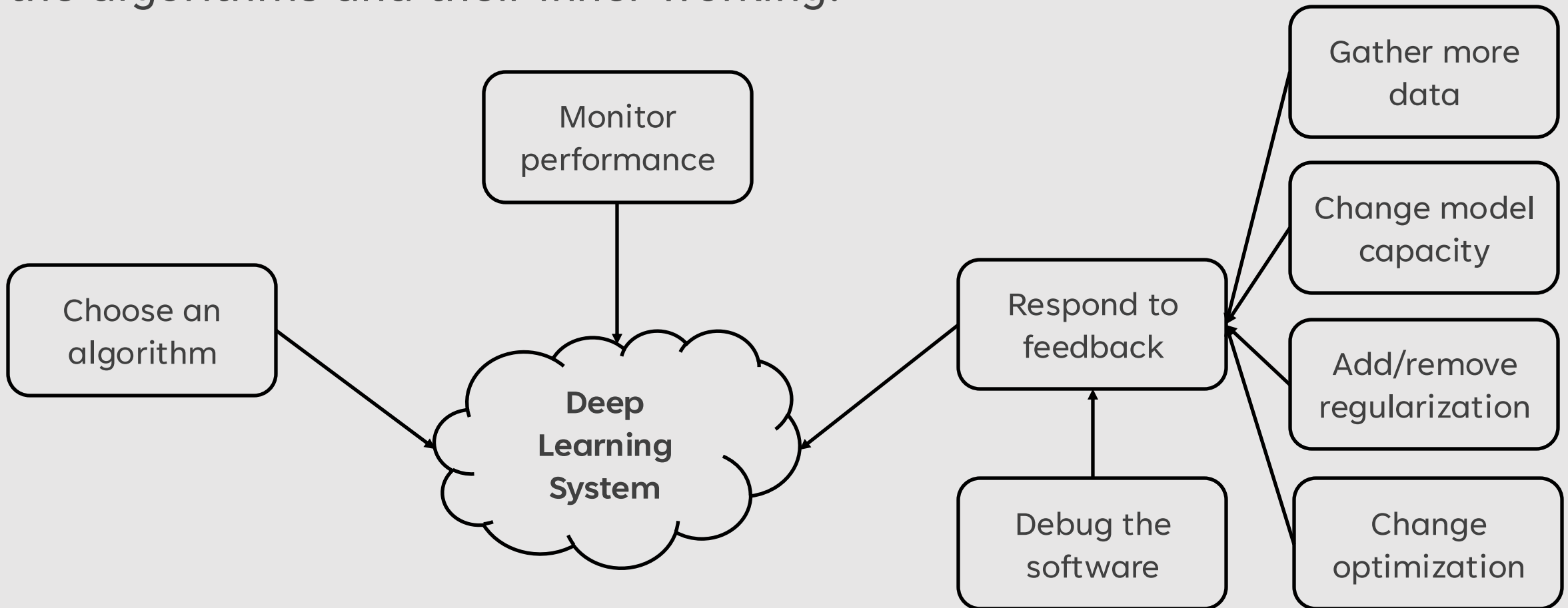
PRACTICAL METHODOLOGY

Successfully applying deep learning requires more than knowledge of the algorithms and their inner working.



PRACTICAL METHODOLOGY

Successfully applying deep learning requires more than knowledge of the algorithms and their inner working.

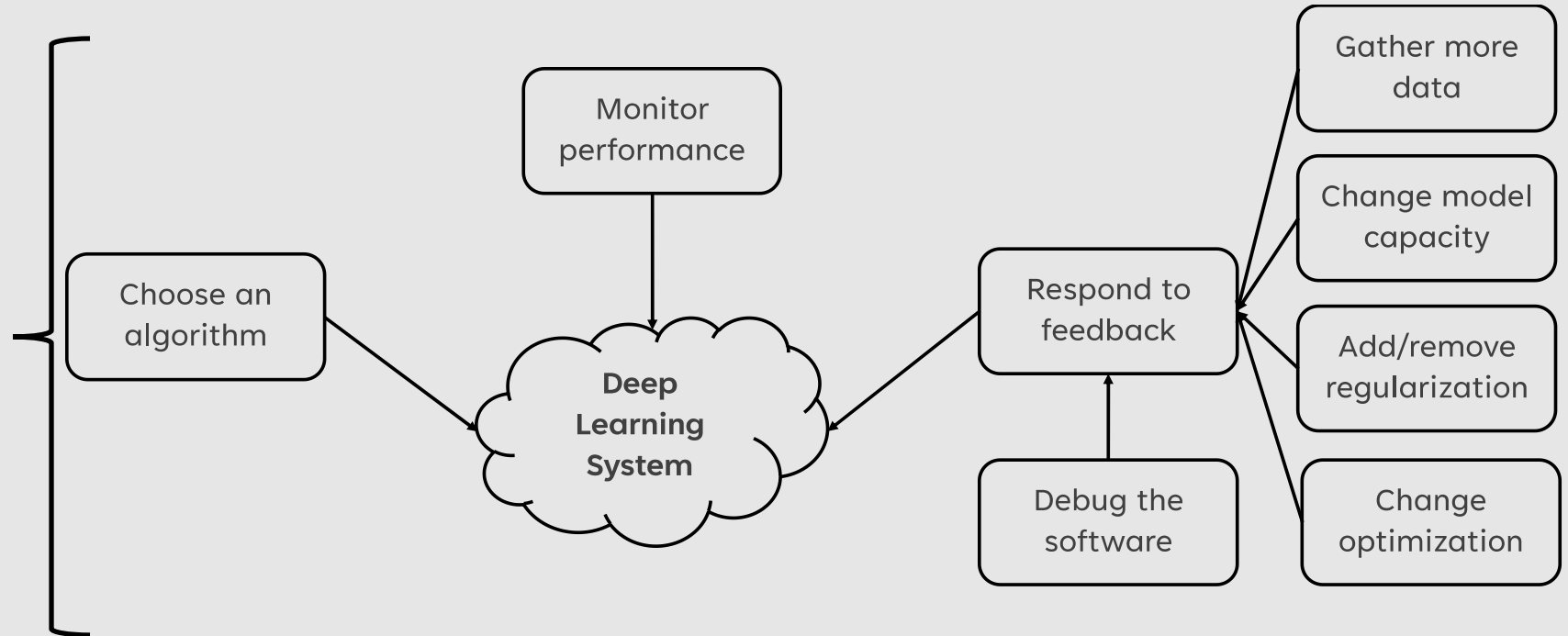


PRACTICAL METHODOLOGY

Successfully applying deep learning requires more than knowledge of the algorithms and their inner working.

All these operations are

- Time consuming
- Costly



It is important to determine in advance the right course of action rather than blindly guessing

PRACTICAL METHODOLOGY

Practical Design Process

1. Determine your goals
 - What error metric to use
 - What the target value for that metric is
 - Metrics and values are driven by the application at hand
2. Establish a working baseline as soon as possible
 - End-to-end pipeline

PRACTICAL METHODOLOGY

Practical Design Process

3. Instrument the system well to determine bottlenecks
 - Which components are performing worse than expected?
 - Why? Overfitting, underfitting, defect in the data, software bug?
4. Make incremental changes based on findings from your instrumentation
 - Add more data
 - Adjust hyperparameters
 - Change gradient based optimization algorithm

PRACTICAL METHODOLOGY – GOALS

Determining the goals is the **first necessary step**:

- The error metric will guide all future actions.
- An idea of the desired level of performance.

Zero error rate might be impossible

- Bayes error: minimum possible error
- Incomplete information about the target variable
- Intrinsically stochastic problems

PRACTICAL METHODOLOGY – GOALS

The error metric **is application dependent**

- Regression
 - MSE
 - MAE
 - R-squared
- Classification
 - Accuracy
 - Precision/Recall
 - Precision-Recall curve

PRACTICAL METHODOLOGY – GOALS

The error metric is **application dependent**

- Regression
 - MSE
 - MAE
 - R-squared
- Classification
 - Accuracy
 - Precision/Recall
 - Precision-Recall curve

Determine the performance metric
ahead of time, then concentrate on
improving this metric.

PRACTICAL METHODOLOGY – GOALS

Determining the **desired level** of performance

- **Academia**
 - Previous published studies
 - State-of-the-art benchmark results
- **Industry**
 - Safe
 - Cost-effective
 - Appealing to consumers

PRACTICAL METHODOLOGY – BASELINES

Establish a **reasonable end-to-end** system:

- Choose a **general category** of a model based on the structure of the data
 - Vector input -> feedforward network
 - Topological structured input -> convolutional networks
 - Sequential input/output -> LSTMs, Transformers

PRACTICAL METHODOLOGY – BASELINES

Establish a **reasonable end-to-end** system:

- Choose a **general category** of a model based on the structure of the data
 - Vector input -> feedforward network
 - Topological structured input -> convolutional networks
 - Sequential input/output -> LSTMs, Transformers
- Begin with **piecewise linear** activation functions
 - ReLUs and their generalizations

PRACTICAL METHODOLOGY – BASELINES

Establish a **reasonable end-to-end** system:

- Choose a **general category** of a model based on the structure of the data
 - Vector input -> feedforward network
 - Topological structured input -> convolutional networks
 - Sequential input/output -> LSTMs, Transformers
- Begin with **piecewise linear** activation functions
 - ReLUs and their generalizations
- Choose an **optimization** algorithm
 - SGD with momentum and decaying learning rate
 - Adam

PRACTICAL METHODOLOGY – BASELINES

Establish a **reasonable end-to-end** system:

- If the dataset is **not** large enough, then
 - Add mild forms of regularization (dropout)
 - Early stopping should be used almost universally

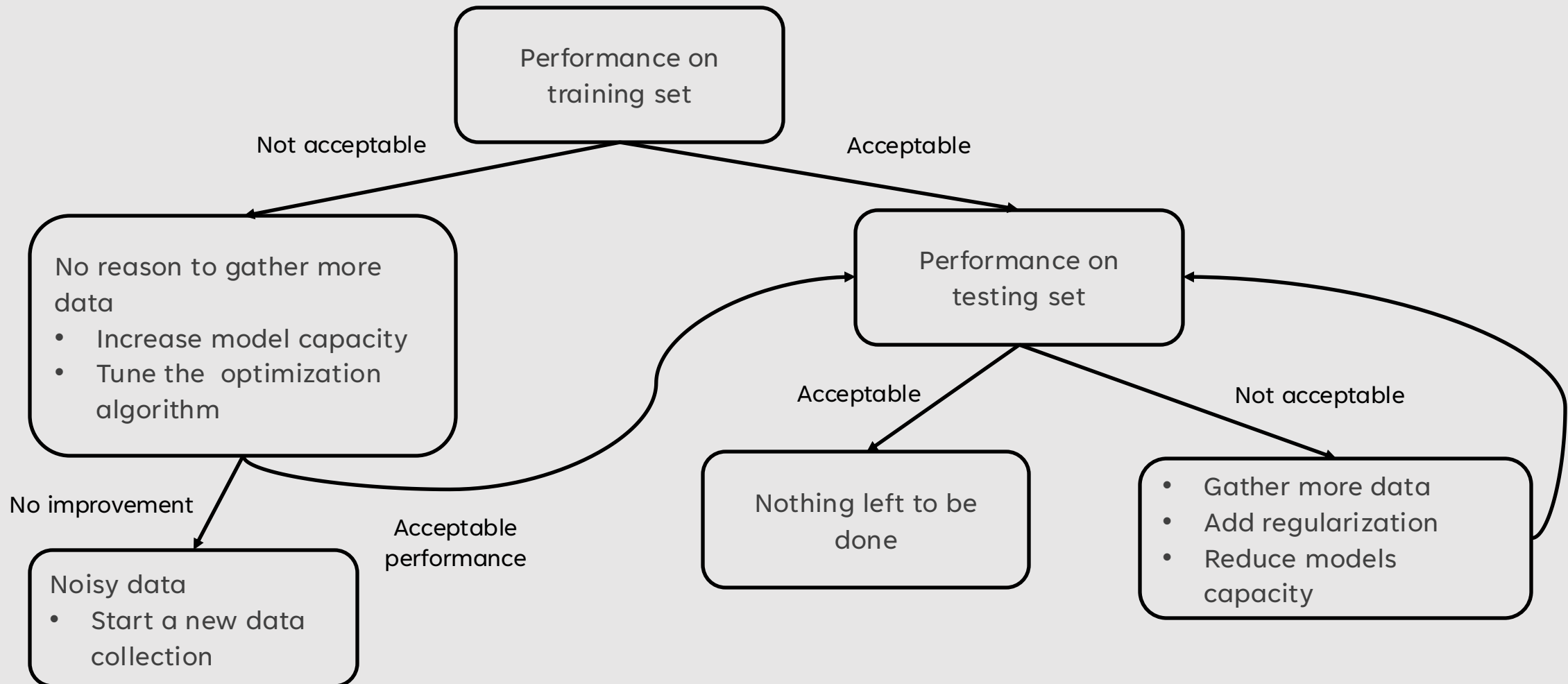
PRACTICAL METHODOLOGY – BASELINES

Establish a **reasonable end-to-end** system:

- If the dataset is **not** large enough, then
 - Add mild forms of regularization (dropout)
 - Early stopping should be used almost universally
- If the task is similar to another task that has been **studied extensively** then
 - Start by copying the model and algorithm that is known to perform best
 - State-of-the-art methodology

PRACTICAL METHODOLOGY – DATA

Determining whether to **gather** more data



PRACTICAL METHODOLOGY – HYPERPARAMETERS

Deep learning algorithms come with many hyperparameters that **control their behaviour**. Hyperparameters also affect

- Performance
- Memory cost of the algorithm
- Time for running the algorithm

PRACTICAL METHODOLOGY – HYPERPARAMETERS

Deep learning algorithms come with many hyperparameters that **control their behaviour**. Hyperparameters also affect

- Performance
 - Memory cost of the algorithm
 - Time for running the algorithm
- } Hyperparameters should be set based on **computation budget**

PRACTICAL METHODOLOGY – HYPERPARAMETERS

Deep learning algorithms come with many hyperparameters that **control their behaviour**. Hyperparameters also affect

- Memory cost of the algorithm
 - Time for running the algorithm
- } Hyperparameters should be set based on **computation budget**

Two different approaches for setting the hyperparameters

- **Manual** hyperparameter setting
 - Understand what each hyperparameter does
- **Automatic** hyperparameter setting
 - Much more computationally costly

PRACTICAL METHODOLOGY – HYPERPARAMETERS

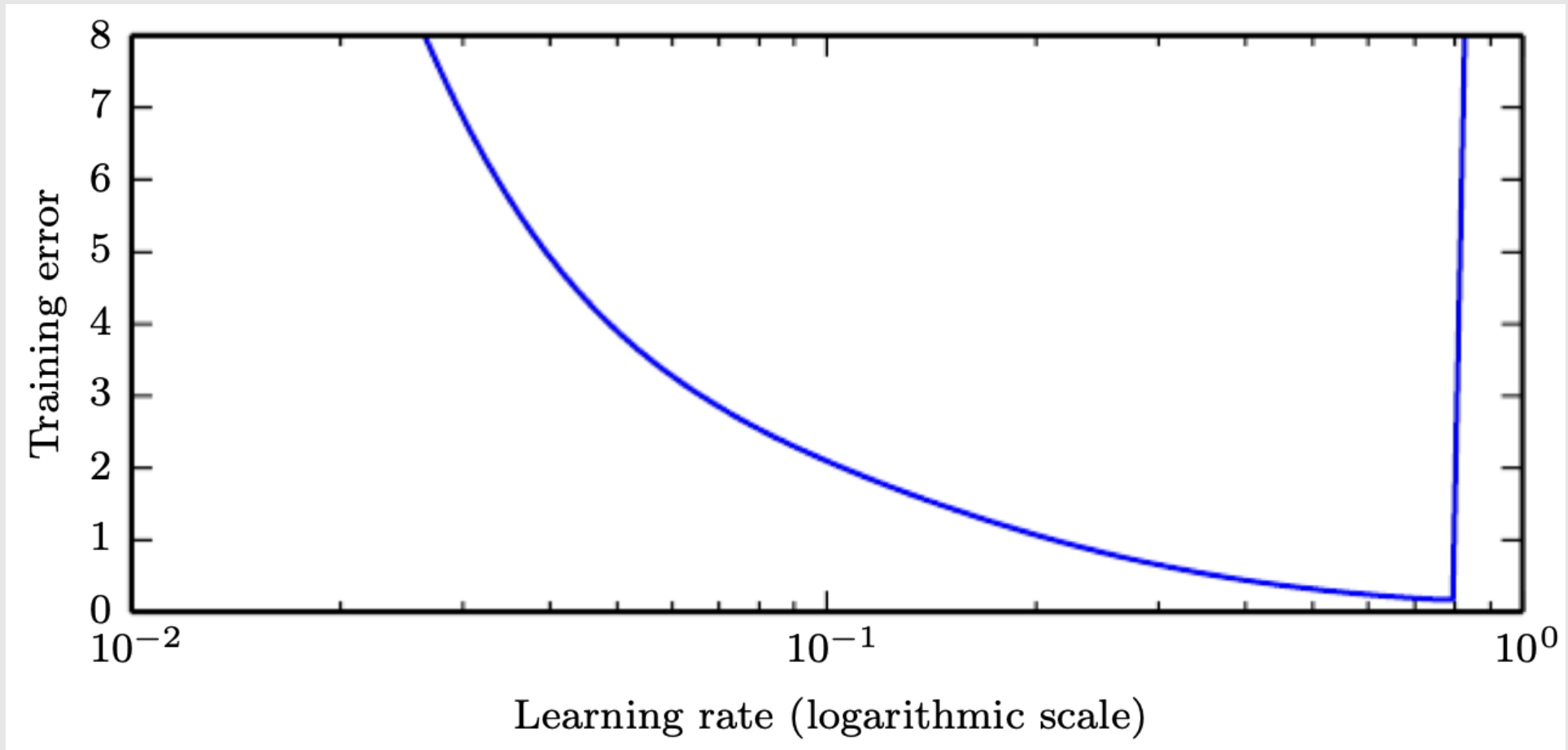
Manual hyperparameter setting requires understanding the interrelationship between:

- Hyperparameters
- Training error
- Generalization error
- Computational resources

Goal: find the lowest generalization error subject to some runtime and memory budget

PRACTICAL METHODOLOGY – HYPERPARAMETERS

Manual hyperparameter setting



PRACTICAL METHODOLOGY – HYPERPARAMETERS

Automatic hyperparameter setting does not require understanding what each hyperparameter does, but it is **computationally intensive**.

Grid search:

1. For each hyperparameter select a small finite set to explore
2. Train a model for each combination of hyperparameter values
3. The best hyperparameters correspond to the model that achieves the best validation performance

PRACTICAL METHODOLOGY – HYPERPARAMETERS

Grid search: How to select a small finite set to explore

- Select the values conservatively
 - Based on prior experience
 - Similar experiments
- Typically, use a logarithmic scale
 - Learning rate {0.1, 0.01, 0.001, 0.0001}
 - Number of hidden units {50, 100, 200, 400, 800, 1600}

PRACTICAL METHODOLOGY – HYPERPARAMETERS

Grid search: How to select a small finite set to explore

- Select the values conservatively
 - Based on prior experience
 - Similar experiments
- Typically, use a logarithmic scale
 - Learning rate {0.1, 0.01, 0.001, 0.0001}
 - Number of hidden units {50, 100, 200, 400, 800, 1600}

The computational cost **grows exponentially** with the number of parameters.

PRACTICAL METHODOLOGY – DEBUGGING

Deep learning systems are **difficult to debug**

- We don't know a priori what the intended behaviour is.
- Multiple components that are adaptive

What can we do?

- Test parts of the deep learning system in isolation
- Develop simple tests
 - Check the gradients
 - Very simple input/output pairs

PRACTICAL METHODOLOGY – DEBUGGING

Debugging strategies:

1. Visualize the model in action
 - Manually evaluate model's predictions for specific inputs
 - Determine whether the quantitative performance numbers are reasonable

PRACTICAL METHODOLOGY – DEBUGGING

Debugging strategies:

1. Visualize the model in action
 - Manually evaluate model's predictions for specific inputs
 - Determine whether the quantitative performance numbers are reasonable
2. Visualise the worst mistakes
 - Incorrect predictions with high confidence
 - Hard examples can reveal problems with the data

PRACTICAL METHODOLOGY – DEBUGGING

Debugging strategies:

1. Visualize the model in action
 - Manually evaluate model's predictions for specific inputs
 - Determine whether the quantitative performance numbers are reasonable
2. Visualise the worst mistakes
 - Incorrect predictions with high confidence
 - Hard examples can reveal problems with the data
3. Reasoning about software using train and test error
 - Errors in models saving or no learning at all

PRACTICAL METHODOLOGY – DEBUGGING

Debugging strategies:

1. Visualize the model in action
 - Manually evaluate model's predictions for specific inputs
 - Determine whether the quantitative performance numbers are reasonable
2. Visualise the worst mistakes
 - Incorrect predictions with high confidence
 - Hard examples can reveal problems with the data
3. Reasoning about software using train and test error
 - Errors in models saving or no learning at all
4. Fit a tiny dataset

EXTRA MATERIAL

<https://www.deeplearningbook.org>

